FORO HUB

API RESTful — Documentación Técnica

Plataforma de foros académicos con autenticación JWT y control de acceso por roles

Versión	1.0.0
Stack	Java 17 · Spring Boot · MySQL · JWT
Repositorio	github.com/nxhuel/foro_hub
API Base URL	`http://localhost:8080/v1`

Tabla de Contenidos

Tabla de Contenidos 2
1. Introducción 4
1.1 Características Principales 4
1.2 Repositorio 4
2. Arquitectura del Sistema 5
2.1 Capas de la Aplicación 5
2.2 Flujo de una Request Autenticada 5
3. Stack Tecnológico 6
4. Modelo de Datos 7
4.1 Diagrama de Entidades 7
4.2 Entidad: Topic 7
4.3 Entidad: User 8
5. Referencia de la API 9
5.0 Visión General (Swagger UI) 9
5.1 Authentication Controller 9
5.2 Topic Controller 10
5.3 Answer Controller 10
5.4 Profile Controller 10
5.5 Course Controller 11
5.6 Role Controller 11
5.7 Permission Controller 12
6. Autenticación y Seguridad 13
6.1 Registro de Usuario (Sign Up) 13
6.2 Inicio de Sesión (Log In) 13
6.3 Uso del Token 13
6.4 Consideraciones de Seguridad 14
7. Guía de Instalación y Configuración 15
7.1 Requisitos del Entorno 15
7.2 Pasos de Instalación 15
Paso 1 — Clonar el Repositorio 15
Paso 2 — Crear la Base de Datos 15
Paso 3 — Configurar application.properties 15
Paso 4 — Ejecutar el Proyecto 15

Paso 5 — Ejecución con Docker (Opcional) 16
8. Paginación de Resultados 17
8.1 Parámetros de Paginación 17
8.2 Ejemplo de Request Paginada 17
9. Códigos de Respuesta HTTP 18
10. Consideraciones y Mejoras Futuras 19
10.1 Mejoras de Seguridad 19
10.2 Calidad y Mantenibilidad 19
10.3 Funcionalidades de Producto 19
11. Glosario 20

1. Introducción

Foro Hub es una API REST backend para la gestión de foros académicos, desarrollada con Java 17 y Spring Boot. El sistema permite a usuarios registrados crear tópicos de discusión, publicar respuestas, y gestionar cursos y perfiles, con un esquema de autenticación stateless basado en JSON Web Tokens (JWT) y control de acceso por roles.

La arquitectura sigue el patrón MVC con separación en capas (Controller → Service → Repository), exponiendo recursos bajo el prefijo /v1 con convenciones REST estándar. La documentación interactiva de la API se genera automáticamente mediante SpringDoc OpenAPI (Swagger UI).

1.1 Características Principales

- Autenticación stateless con JWT (sign-up + log-in)
- CRUD completo de tópicos con paginación de resultados
- Gestión de respuestas por tópico
- Gestión de perfiles de usuario
- Gestión de cursos académicos
- Sistema de roles y permisos (RBAC)
- Documentación interactiva con Swagger UI

1.2 Repositorio

Recurso	URL
Repositorio Backend	https://github.com/nxhuel/foro_hub
Swagger UI	http://localhost:8080/swagger-ui.html
OpenAPI JSON	http://localhost:8080/v3/api-docs

2. Arquitectura del Sistema

Foro Hub implementa una arquitectura en capas desacoplada. Cada capa tiene una responsabilidad única y se comunica exclusivamente con la capa adyacente, lo que facilita el testing unitario y la mantenibilidad del código.

2.1 Capas de la Aplicación

Capa	Componente	Responsabilidad
Seguridad	`SecurityFilterChain / JwtFilter`	Intercepción de requests, validación JWT, carga del SecurityContext
Presentación	`@RestController`	Exposición de endpoints REST, validación de DTOs de entrada, serialización JSON
Negocio	`@Service`	Lógica de dominio, validaciones de negocio, mapeo de entidades a DTOs
Persistencia	`@Repository / JPA`	Acceso a datos, queries JPQL, transacciones
Base de Datos	`MySQL 8.x`	Almacenamiento relacional, integridad referencial, índices

2.2 Flujo de una Request Autenticada

Toda request a un endpoint protegido sigue este flujo antes de llegar al controller:

- Cliente envía el token JWT en el header: Authorization: Bearer <token>
- El JwtFilter intercepta la request y extrae el token del header
- Se valida la firma, expiración y estructura del JWT
- Se carga el UserDetails desde la base de datos con el username del token
- Se establece el Authentication en el SecurityContextHolder
- La request pasa al controller correspondiente ya autenticada

3. Stack Tecnológico

Tecnología	Versión / Tipo	Propósito
Java	JDK 17+	Lenguaje principal del backend
Spring Boot	Framework	Framework principal: REST, seguridad, inyección de dependencias
Spring Security	Módulo	Autenticación JWT y autorización basada en roles
JWT (JJWT)	Librería	Generación y validación de JSON Web Tokens
JPA / Hibernate	ORM	Mapeo objeto-relacional y gestión de transacciones
MySQL	RDBMS	Motor de base de datos relacional
SpringDoc OpenAPI	3.x	Generación automática de Swagger UI y especificación OpenAPI 3.0
Maven	3.8+	Gestión de dependencias y ciclo de vida del build
Docker	Opcional	Containerización para despliegue reproducible

4. Modelo de Datos

La base de datos está estructurada en seis entidades principales interrelacionadas. El diseño normaliza la información evitando redundancia y garantizando integridad referencial mediante claves foráneas.

4.1 Diagrama de Entidades

Entidad	Tabla DB	Relaciones	Descripción
User	`users`	Tiene roles, perfiles, tópicos y respuestas	Usuario del sistema con credenciales
Topic	`topics`	Pertenece a un User y a un Course	Hilo de discusión creado por un usuario
Answer	`answers`	Pertenece a un Topic y a un User	Respuesta dentro de un tópico
Course	`courses`	Agrupa tópicos de una misma materia	Curso o materia académica
Profile	`profiles`	Extensión de datos del User (1:1)	Información adicional del usuario
Role	`roles`	Asignado a uno o más Users	Rol de acceso (ADMIN, USER, etc.)

4.2 Entidad: Topic

Entidad central del sistema. Representa un hilo de discusión dentro de un curso. Soporta paginación en los endpoints de consulta.

Campo	Tipo	Restricciones	Descripción
`id`	BIGINT	PK, AUTO_INCREMENT	Identificador único del tópico
`title`	VARCHAR(255)	NOT NULL	Título descriptivo del tópico
`message`	TEXT	NOT NULL	Contenido del mensaje principal
`created_at`	DATETIME	NOT NULL	Fecha y hora de creación (UTC)
`status`	ENUM	DEFAULT 'OPEN'	Estado: OPEN, CLOSED, DELETED
`author_id`	BIGINT	FK -> users.id	Usuario que creó el tópico
`course_id`	BIGINT	FK -> courses.id	Curso al que pertenece el tópico

4.3 Entidad: User

Campo	Tipo	Restricciones	Descripción
`id`	BIGINT	PK, AUTO_INCREMENT	Identificador único del usuario
`username`	VARCHAR(100)	UNIQUE, NOT NULL	Nombre de usuario para autenticación
`password`	VARCHAR(255)	NOT NULL	Contraseña encriptada con BCrypt
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Correo electrónico del usuario
`enabled`	BOOLEAN	DEFAULT true	Estado activo/inactivo de la cuenta
`role_id`	BIGINT	FK -> roles.id	Rol asignado al usuario (RBAC)

5. Referencia de la API

Todos los endpoints se encuentran bajo el prefijo /v1. Los endpoints marcados con JWT requieren el header Authorization: Bearer <token>. Los endpoints de autenticación (/v1/auth/*) son públicos.

5.0 Visión General (Swagger UI)

Figura 1: Swagger UI — Listado completo de controladores y endpoints del sistema

5.1 Authentication Controller

Endpoints públicos para registro e inicio de sesión. En el sign-up se genera el usuario con contraseña encriptada; en el log-in se retorna el JWT firmado para usar en los demás endpoints.

Método	Endpoint	Auth	Descripción

POST	/v1/auth/sign-up	Público	Registrar nuevo usuario. Body: { username, email, password, roleId }
POST	/v1/auth/log-in	Público	Autenticar usuario. Body: { username, password }. Retorna JWT.

5.2 Topic Controller

Gestión completa de tópicos. Los endpoints de escritura (POST, PUT, DELETE) requieren autenticación JWT. La consulta de listado soporta paginación mediante parámetros page y size.

Método	Endpoint	Auth	Descripción
GET	/v1/topics	JWT	Listar todos los tópicos. Parámetros: page, size, sort.
GET	/v1/topic/{topicId}	JWT	Obtener detalle de un tópico por ID.
GET	/v1/topic-answer/{topicId}	JWT	Obtener tópico con todas sus respuestas.
POST	/v1/create-topic	JWT	Crear nuevo tópico. Body: { title, message, courseId }.
PUT	/v1/update-topic/{topicId}	JWT	Actualizar tópico existente. Solo el autor puede editar.
DELETE	/v1/delete-topic/{topicId}	JWT	Eliminar tópico por ID. Solo el autor o un ADMIN.

5.3 Answer Controller

Gestión de respuestas dentro de un tópico. Cada respuesta está vinculada a un tópico padre y al usuario que la publicó.

Método	Endpoint	Auth	Descripción
POST	/v1/create-answer	JWT	Publicar respuesta en un tópico. Body: { message, topicId }.
PUT	/v1/update-answer/{answerId}	JWT	Editar una respuesta. Solo el autor puede modificarla.
DELETE	/v1/delete-answer/{answerId}	JWT	Eliminar una respuesta. Autor o ADMIN.

5.4 Profile Controller

Gestión de perfiles de usuario. Cada usuario tiene exactamente un perfil asociado (relación 1:1).

Método	Endpoint	Auth	Descripción
GET	/v1/profiles	JWT	Listar todos los perfiles de usuario.
GET	/v1/profile/{profileId}	JWT	Obtener perfil por ID.
POST	/v1/create-profile	JWT	Crear perfil para el usuario autenticado.
PUT	/v1/update-profile/{profileId}	JWT	Actualizar datos del perfil.
DELETE	/v1/delete-profile/{profileId}	JWT	Eliminar perfil por ID.

5.5 Course Controller

Gestión de cursos académicos. Los cursos agrupan tópicos relacionados a una misma materia o disciplina.

Método	Endpoint	Auth	Descripción
GET	/v1/courses	JWT	Listar todos los cursos disponibles.
GET	/v1/course/{courseId}	JWT	Obtener detalle de un curso por ID.
POST	/v1/create-course	JWT	Crear nuevo curso. Body: { name, description }.
DELETE	/v1/delete-course/{courseId}	JWT	Eliminar curso. Solo ADMIN.

5.6 Role Controller

Gestión de roles del sistema. Las operaciones sobre roles están reservadas para usuarios con rol ADMIN.

Método	Endpoint	Auth	Descripción
GET	/v1/roles	JWT	Listar todos los roles disponibles en el sistema.
POST	/v1/create-role	JWT	Crear nuevo rol. Body: { name }. Solo ADMIN.
DELETE	/v1/delete-role/{roleId}	JWT	Eliminar rol por ID. Solo ADMIN.

5.7 Permission Controller

Gestión de permisos granulares. Los permisos se asignan a roles para controlar el acceso a funcionalidades específicas.

Método	Endpoint	Auth	Descripción
GET	/v1/permissions	JWT	Listar todos los permisos registrados.
POST	/v1/create-permission	JWT	Crear nuevo permiso. Solo ADMIN.
DELETE	/v1/delete-permission/{permissionId}	JWT	Eliminar permiso. Solo ADMIN.

6. Autenticación y Seguridad

6.1 Registro de Usuario (Sign Up)

Para registrar un nuevo usuario se realiza un POST a /v1/auth/sign-up con el siguiente body JSON:

```
POST /v1/auth/sign-up
Content-Type: application/json
```

```
{
  "username": "johndoe",
  "email":    "john@example.com",
  "password": "miPassword123",
  "roleId":   1
}
```

La contraseña se almacena encriptada con BCrypt. No se retorna el password en ninguna respuesta del sistema.

6.2 Inicio de Sesión (Log In)

Para obtener un token JWT se realiza un POST a /v1/auth/log-in:

```
POST /v1/auth/log-in
Content-Type: application/json
```

```
{
  "username": "johndoe",
  "password": "miPassword123"
}
```

Respuesta exitosa (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

6.3 Uso del Token

Para consumir endpoints protegidos, incluir el token JWT en el header de cada request:

```
GET /v1/topics
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

El token tiene un tiempo de expiración configurable en application.properties. Una vez vencido, se debe obtener uno nuevo mediante un nuevo log-in.

6.4 Consideraciones de Seguridad

- Los passwords se almacenan con BCrypt (factor de costo 10 por defecto)
- El JWT se firma con una clave secreta configurada en variables de entorno
- Los endpoints sensibles requieren rol ADMIN (gestión de roles y permisos)
- Se aplica CORS configurado para el dominio del frontend autorizado
- Las contraseñas nunca se retornan en respuestas de la API

7. Guía de Instalación y Configuración

7.1 Requisitos del Entorno

Componente	Versión Mínima	Notas
JDK	17+	Se recomienda JDK 21 LTS
MySQL Server	8.0+	Crear schema 'foro_hub' previamente
Maven	3.8+	O usar Maven Wrapper incluido: ./mvnw
Docker (opcional)	20.10+	Para despliegue containerizado

7.2 Pasos de Instalación

Paso 1 — Clonar el Repositorio

```
git clone https://github.com/nxhuel/foro_hub.git
cd foro_hub
```

Paso 2 — Crear la Base de Datos

```
CREATE DATABASE foro_hub CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Paso 3 — Configurar application.properties

Editar el archivo src/main/resources/application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/foro_hub
spring.datasource.username=root
spring.datasource.password=tu_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

```
# JWT Configuration
api.security.secret=tu_clave_secreta_larga_y_aleatoria_minimo_32_chars
api.security.expiration=86400000
```

Paso 4 — Ejecutar el Proyecto

```
mvn spring-boot:run
```

La API quedará disponible en http://localhost:8080. Acceder a la documentación interactiva en http://localhost:8080/swagger-ui.html.

Paso 5 — Ejecución con Docker (Opcional)

```
docker build -t foro-hub .
docker run -p 8080:8080 \
  -e DB_URL=jdbc:mysql://host.docker.internal:3306/foro_hub \
  -e DB_USERNAME=root \
  -e DB_PASSWORD=tu_password \
  -e JWT_SECRET=tu_clave_secreta \
  foro-hub
```

8. Paginación de Resultados

El endpoint GET /v1/topics soporta paginación nativa de Spring Data. Los parámetros se envían como query params en la URL.

8.1 Parámetros de Paginación

Parámetro	Tipo	Default	Descripción
`page`	Integer	0	Número de página (base 0)
`size`	Integer	10	Cantidad de elementos por página
`sort`	String	id,asc	Campo y dirección de ordenamiento (ej: created_at,desc)

8.2 Ejemplo de Request Paginada

```
GET /v1/topics?page=0&size=5&sort=created_at,desc
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Respuesta (estructura Page de Spring):

```
{
  "content": [ { "id": 10, "title": "...", ... } ],
  "pageable": { "pageNumber": 0, "pageSize": 5 },
  "totalElements": 47,
  "totalPages": 10,
  "last": false
}
```

9. Códigos de Respuesta HTTP

Código	Estado	Contexto de uso
200	OK	Request exitosa (GET, PUT)
201	Created	Recurso creado exitosamente (POST)
204	No Content	Eliminación exitosa (DELETE)
400	Bad Request	Body inválido o parámetros incorrectos
401	Unauthorized	Token JWT ausente, inválido o expirado
403	Forbidden	El usuario no tiene permisos para la acción
404	Not Found	Recurso no encontrado por el ID especificado
409	Conflict	Username o email ya registrado (sign-up)
500	Internal Server Error	Error interno no controlado del servidor

10. Consideraciones y Mejoras Futuras

10.1 Mejoras de Seguridad

- Implementar refresh tokens para renovación de JWT sin re-autenticación
- Agregar rate limiting por IP para prevenir ataques de fuerza bruta en /v1/auth/log-in
- Implementar logout con blacklist de tokens invalidados (Redis recomendado)
- Agregar validación de complejidad de contraseñas en el registro

10.2 Calidad y Mantenibilidad

- Agregar Flyway o Liquibase para versionado de migraciones de base de datos
- Implementar tests unitarios con JUnit 5 y Mockito para la capa de servicios
- Agregar tests de integración con @SpringBootTest y base de datos H2 en memoria
- Incorporar borrado lógico (soft delete) con campo deleted_at en tópicos y respuestas
- Aplicar DTOs de respuesta diferenciados de los de entrada para mayor control del contrato

10.3 Funcionalidades de Producto

- Implementar búsqueda full-text de tópicos por título y contenido
- Agregar sistema de notificaciones cuando se responde un tópico propio
- Incorporar marcado de respuesta como 'solución aceptada' por el autor del tópico
- Implementar votación (upvote/downvote) en tópicos y respuestas

11. Glosario

Término	Definición
JWT	JSON Web Token. Token firmado digitalmente para autenticación stateless. Contiene claims (datos) del usuario codificados en Base64.
BCrypt	Algoritmo de hashing adaptativo para contraseñas. Incluye salt automático y es resistente a ataques de fuerza bruta.
RBAC	Role-Based Access Control. Modelo de autorización donde los permisos se asignan a roles y los roles a usuarios.
Stateless	Arquitectura donde el servidor no almacena estado de sesión. Cada request es autosuficiente y lleva su propio token.
Paginación	Mecanismo para dividir resultados grandes en páginas. Evita traer todos los registros de la BD en una sola query.
DTO	Data Transfer Object. Objeto plano para transportar datos entre capas, desacoplando el modelo de dominio de la API.
OpenAPI / Swagger	Especificación estándar para documentar APIs REST. Swagger UI genera una interfaz gráfica interactiva a partir de anotaciones en el código.
SecurityContext	Objeto de Spring Security que almacena la autenticación del usuario actual durante el ciclo de vida de una request.

Foro Hub — API REST Documentation v1.0.0

github.com/nxhuel/foro_hub